

词嵌入 Word2vec

- 词向量：用于表示单词意义的向量
- 词嵌入：将单词映射到向量的技术

为何抛弃独特编码

主要原因：不能准确表达不同词之间的相似度

以余弦相似度为例，独热编码的相似度恒为0：

$$\frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \in [-1, 1].$$

自监督的word2vec

包含两个模型：

- Skip-Gram
 - 给定中间词，预测周围上下文
- CBOW：continuous bag-of-word
 - 给定周围上下文，预测中间词

训练依赖于条件概率，可以看做使用一些词预测另一些词，由于词没有标签，所以可以看作是自监督模型

近似训练

在自监督Word2Vec中，在词典上求和的梯度往往计算量很大。为了减少计算量，可以进行

- 负采样
 - 只筛选出正样本（待预测的）和少量负样本（干扰项）用于训练
- 层序softmax
 - 构造一个二叉树，每个词对应叶子节点，从线性搜索变成对数搜索
 - 路径长度 = $\log_2(\text{词典大小})$

子词嵌入

不把每个词当整体，而是把词拆成更小的单元来表示。例如，当训练时见到help, helped, helping，能在测试时说出helps大致意思。子词嵌入可以提高稀有词和词典外词的表示质量。有两类方法

fastText模型

- Word2vec中，同一词的不同形式由不同向量组成
- fastText中，字词是一个字符n-gram，类似于Skip-Gram，每个中心词由子词级向量之和表示

以 "where" 为例：

1. 在开头添加 "<" 和 ">"，得到 "<where>"
2. 从词中提取字符 n-gram，以n=3为例，获得
 - i. 长度为3的子词 "<wh" "whe" "her" "ere" "re>"
 - ii. 特殊子词 "<where>"

在fastText中，对于任意词 w ，用 $\mathcal{G}_w$ 表示其长度在3和6之间的所有子词与其特殊子词的并集。词表是所有词的子词的集合。假设 $\mathbf{z}_g$ 是词典中的子词 g 的向量，则跳元模型中作为中心词的词 w 的向量 $\mathbf{v}_w$ 是其子词向量的和：

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g.$$

字节对编码 Byte Pair Encoding BPE

fastText中，词表大小不能预定义。为了在固定大小词表中允许可变长度的子词，引入BPE，一种 **数据驱动** 的子词切分方法，从数据中统计学习，以发现词内的公共符号，是一种贪心方法

1. 符号词表初始化为26个字母 + 特殊词尾符号 "_" + 特殊未知符号 "[UNK]"
2. 引入初始数据，里面有 "fast_" "faster_" "tall_" "taller_"
3. 进行迭代，发现最频繁的符号对
 - i. 如 t + a ，迭代后合并得到新符号 —— "ta"
 - ii. 如 ta + l ，迭代后合并得到新符号 —— "tal"
 - iii. 反复进行迭代，直到达到上限

来自Transformer的双向编码器表示 BERT

从上下文无关到上下文敏感

词嵌入模型与上下文无关，具有明显的局限性，推动了上下文敏感 $f(x, c(x))$ 的发展，其中 x 是token， $c(x)$ 为 x 的上下文，典型上下文敏感如ELMo，将整个序列作为输入，为序列内每个单词分配一个表示的函数。

但是ELMo虽好，但只服务于特定任务

从特定于任务到不可知任务

GPT(Generative Pre Training, 生成式预训练)在Transformer Decoder的基础上，预训练一个用于表示文本序列的语言模型。

但是，由于语言模型的自回归特性，即使用自己过去的词预测自己未来的词，只能从左向右看，在部分语境中表现很差。

BERT：集大成者

- ELMo对上下文双向编码，但只服务于特定任务
- GPT与任务无关，但只从左到右编码上下文

BERT：Bidirectional Encoder Representations from Transformers，结合了两者的优点

- 对上下文双向编码
- 仅需对架构少量改变即可适用于大多数NLP任务

其中，在下游任务监督学习中，与GPT有两点类似：

- BERT表示(相当于词向量)被输入到新添加的输出层中，同时根据任务的性质（如分类或标注等）对架构进行最小更改
- 对预训练后的 Transformer Encoder 所有参数进行微调；新添加的输出层额外训练

 ELMo、GPT和BERT的比较

输入表示

1. **输入序列，即原始token的排列**：在NLP中，任务不同，输入序列的格式也有所不同

- 有些以 **单个文本** 作为输入
 - 输入序列 = [CLS] + 文本 + [SEP]
- 有的以 **一堆文本序列** 作为输入
 - 输入序列 = [CLS] + 文本A + [SEP] + 文本B + [SEP]

2. **词嵌入**

- 字面意思，将输入序列排好后就开始词嵌入

3. **片段嵌入**

- 为了区分文本对，可以用片段嵌入 e_A 和 e_B 添加到第一序列和第二序列的**词嵌入**中
- 而单文本则使用 e_A

4. **位置嵌入**

- 与Transformer不同，该位置嵌入可学习

最终，输入到BERT的是 词嵌入 + 段嵌入 + 位置嵌入

 BERT输入序列的嵌入是词元嵌入、片段嵌入和位置嵌入的和

预训练任务

掩蔽语言模型：让BERT学会词级理解

为了双向编码上下文以表示token，BERT随机掩蔽token并使用来自双向上下文来预测掩蔽token，该自监督的任务称作掩蔽语言模型

以 "the movie is great" 为例，随机选择少量token为预测的掩蔽token，这里选择 "great" ，为了掩蔽，通常使用 "[MASK]" 来替换掉 great ，然而 "[MASK]" 不会出现在微调中，所以有以下情况：

- 大部分：掩蔽token，如[MASK]
- 少部分：随机token，如drink
- 少部分：内容不变，如great

下一句预测：让BERT学会句间关系

一个二分类任务，用于理解句子之间逻辑关系，判断是否相邻。预训练时，有一半的正样本和一半的负样本，BERT通过[CLS]的输出向量，将[CLS]编码后送入二分类线性层，判断是否相邻

微调BERT

可以应用于以下任务

- 单文本分类
 - 如情感分析
- 文本对分类或回归
 - 回归的典型例子是判断句子之间的语义相似度
- 文本标注
 - 例如标注词性
- 问答